

From QAT to GRPO: A Two-Stage FP4 LLM Post-Training Pipeline with Causal Moving-Mean Quantization

Anonymous submission

Abstract

Low-precision training is attractive for large-language-model post-training, but reinforcement-learning-based post-training introduces a distinct failure mode: the policy must generate its own training data. We find that directly replacing the linear layers of a BF16 policy with an NVFP4 simulation can corrupt early rollouts through repetition, mixed-language fragments, and prematurely terminated responses. These failures reduce the reliability of group-relative reward signals and prevent stable policy optimization. We present a post-training pipeline that first adapts the quantized policy by self-distillation from its BF16 counterpart and then applies Group Relative Policy Optimization (GRPO) using a causal moving-mean residual quantization path. The implementation includes rollout-quality and distribution diagnostics that make generation failures measurable during training. Preliminary controlled runs show substantially lower rule-based bad-rollout rates and higher rewards after quantization-aware distillation than after direct FP4 GRPO.

Introduction

Post-training aligns a pretrained language model with task objectives through supervised, preference, or reinforcement-learning signals. For reasoning models, reinforcement-learning (RL) post-training is particularly effective because it improves a policy using rewards assigned to its own sampled solutions. Modern policy-gradient objectives such as PPO and its group-relative variant GRPO make this optimization practical for large language models (Schulman et al. 2017; Shao et al. 2024b). Yet this capability is expensive: the policy must repeatedly store activations, generate multiple long completions per prompt, and evaluate their rewards. FP4 arithmetic offers an attractive route to reducing the memory and compute cost of this process, and recent FP4 systems have demonstrated that carefully designed quantization can support large-scale training and efficient inference (NVIDIA et al. 2025; Huang et al. 2025).

RL post-training nevertheless poses a challenge that is absent from fixed-batch training and deployment-time quantization. The current policy is also a data generator: it produces

candidate completions, which are scored and used to update that same policy. This closed loop amplifies small perturbations in the quantized forward path. A quantization error that only modestly changes logits can change a sampled completion, alter group-relative rewards, and shift the next policy update. The relevant question is therefore not only whether an FP4 model attains low loss on teacher-forced tokens, but whether it can generate the diverse, interpretable rollouts required to construct a useful RL learning signal.

We study the following question: *how can an FP4 policy be made reliable enough to generate the rollouts required for RL post-training?* Our starting observation is empirical and operational. Directly enabling the FP4 path for a BF16 policy before GRPO produces nonsensical repetitions, mixed-language fragments, and very short outputs early in training. The problem is not explained by visible numerical overflow: the diagnostic runs report no NaN or Inf events and stable activation-magnitude summaries. Instead, degraded decoding yields low-information groups of responses, weakening the relative-reward signal on which GRPO relies.

Our solution separates quantization adaptation from reward optimization. Before GRPO, a BF16 teacher samples responses and supplies response-token logits to an FP4 student. The student is optimized with a masked distillation objective on the same responses that define the teacher’s decoding distribution. The adapted student then enters GRPO with the same quantized forward path. To preserve the benefit of mean-residual quantization during causal generation, we maintain a token-counted moving mean after prompt prefill and update it at each decode step. This two-stage design targets the source of the instability—the rollout distribution—before policy optimization can amplify it.

This work makes the following contributions:

- a rollout-centric diagnosis of direct-FP4 GRPO failure, supported by quantitative text-quality and training-signal diagnostics;
- a QAT self-distillation stage that adapts an FP4 policy before reward optimization;
- a causal moving-mean residual FP4 implementation for both training and autoregressive decode; and
- a matched evaluation protocol spanning mathematical reasoning and general reasoning benchmarks.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Related Work

FP4 Quantized Training

Training with FP4 is challenging because blockwise scales must represent both rare high-magnitude values and the much denser bulk of a tensor. Metis attributes this difficulty to spectral anisotropy and uses structured decomposition to quantize narrower sub-distributions (Cao et al. 2025). Averis identifies a coherent activation mean as an important source of outliers and quantizes a centered residual (Cao et al. 2026). Other FP4-training systems combine mixed-precision optimization, differentiable quantization, and outlier compensation to stabilize large-scale pretraining (Wang et al. 2025; Chmiel et al. 2025). NVIDIA’s NVFP4 pretraining recipe combines random Hadamard transforms, two-dimensional quantization, stochastic rounding, and selective high-precision layers (NVIDIA et al. 2025). In the post-training setting, QeRL combines NVFP4 weight quantization with LoRA and adaptive quantization noise to reduce rollout cost while optimizing reasoning policies (Huang et al. 2025).

Post-Training Quantization for LLM Inference

Post-training quantization (PTQ) converts a trained model using calibration data, without replaying its original optimization procedure. Early LLM PTQ work identified activation outliers as a central obstacle (Dettmers et al. 2022); SmoothQuant shifts the activation-quantization burden into weights through an equivalent transformation (Xiao et al. 2023). Data-aware weight-only methods use approximate second-order information or activation-aware scaling to preserve important directions (Frantar et al. 2023; Lin et al. 2024). Transformation-based methods reduce activation and weight outliers before quantization: QuaRot applies invariant rotations to hidden states, feed-forward activations, attention components, and KV caches (Ashkboos et al. 2024), whereas FlatQuant learns affine transformations that flatten layerwise weight and activation distributions (Sun et al. 2024). OmniQuant instead learns clipping and equivalent transformations during calibration (Shao et al. 2024a). LLM-FP4 studies floating-point PTQ through format and clipping-range search together with per-channel activation scaling (Liu et al. 2023); QRazor instead retains salient bits from an integer representation to quantize weights, activations, and KV caches without additional training (Lee, Choi, and Chang 2025). These methods target a fixed, deployment-time quantizer and inference accuracy. They do not address the coupled rollout-and-update process in which an RL policy generates the trajectories used for its own optimization.

Quantization-Aware Adaptation for Post-Training

Quantization-aware training (QAT) adapts a quantized forward path through fine-tuning, while quantization-aware distillation (QAD) further aligns a quantized student with a high-precision teacher. Classical QAT learns quantizer parameters directly against task loss (Esser et al. 2020), and knowledge distillation provides the standard teacher–student distribution-matching objective (Hinton, Vinyals, and Dean 2015). For LLMs, prior work has combined self-distillation

with sub-4-bit QAT (Du et al. 2024). NVIDIA recently introduced QAD for NVFP4 inference-accuracy recovery after complex post-training pipelines, including supervised fine-tuning and RL, using a KL-divergence objective between a full-precision teacher and a quantized student (Xin et al. 2026). Its reported NVFP4 configurations retain selected Transformer modules and boundary layers in BF16, and the objective is post-hoc inference recovery rather than on-policy reward optimization. This line of work establishes distillation as a practical way to recover low-precision accuracy for post-trained models.

Motivation. Taken together, existing FP4 training methods establish mechanisms for controlling quantization error during pretraining, and PTQ methods optimize a fixed model for inference. Quantization-aware adaptation recovers low-precision accuracy after a post-training pipeline, but it does not directly address the transition from a BF16 policy to an FP4 policy that must immediately generate on-policy trajectories. RL post-training instead couples quantization, decoding, reward construction, and policy updates in a single feedback loop. This setting motivates a rollout-centric FP4 pipeline: first adapt the quantized computation path to a stable teacher distribution, then optimize the adapted policy with RL while preserving a causal quantization state during decoding.

Method

FP4 Post-Training with GRPO

For a prompt x , GRPO samples a group of completions $\{y_i\}_{i=1}^G$ from the current policy π_θ . A task reward $r(x, y_i)$ is converted into a group-relative advantage, and the policy is updated with a clipped objective. Consequently, a useful optimization step presupposes that the sampled group contains interpretable responses with reward variation. If quantized decoding produces repeated or malformed text, relative comparisons can become noisy even when the reward function itself is correctly implemented.

Figure 1 provides an overview of the proposed two-stage pipeline before the component details below. It first adapts an FP4 student to a BF16 teacher through quantization-aware self-distillation, then uses the adapted policy for GRPO with causal moving-mean residual quantization.

More concretely, let $\hat{A}_i$ denote the reward advantage normalized within a group of G completions, and let $\rho_i(\theta)$ be the ratio between the current and rollout-policy probabilities of y_i . The policy term takes the standard clipped form

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \min(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i). \quad (1)$$

Thus, the relevant failure is not merely a larger quantization error on a fixed batch: malformed samples can reduce both the semantic validity of $r(x, y_i)$ and the variation required to form useful within-group advantages.

The teacher and the BF16 reference policy use BF16 linear layers. For the FP4 policy, trainable master parameters

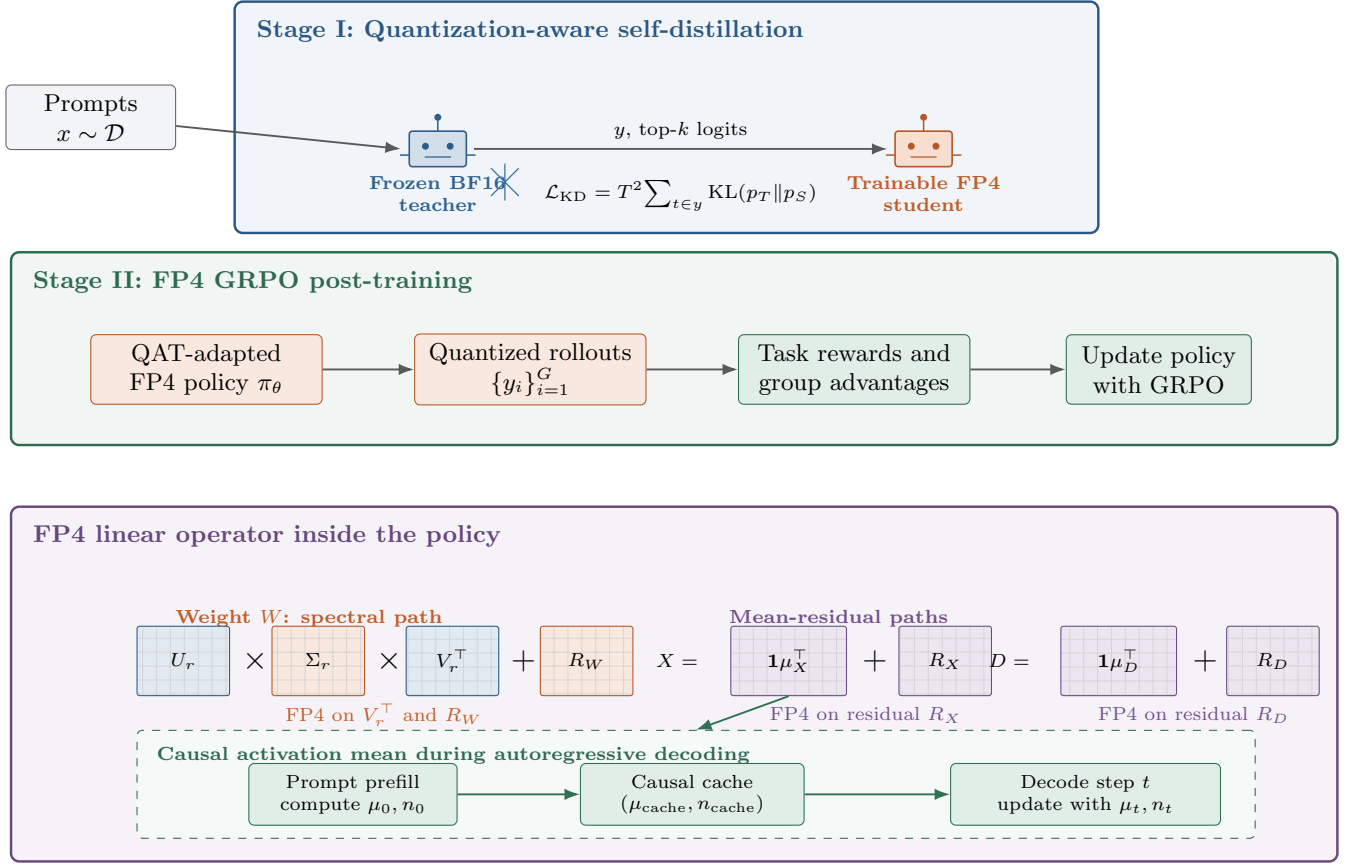


Figure 1: Overview of FP4 post-training. In Stage I, the ice-blue teacher robot and snowflake denote frozen BF16 parameters, whereas the warm FP4 student robot is updated by masked token-level distillation. Stage II uses the adapted FP4 policy to generate GRPO rollouts and updates it using group-relative rewards. The grid blocks visualize the fixed weight decomposition $U_r \Sigma_r V_r^\top + R_W$ and the mean-residual splits for activations and output gradients; only the residual matrices use FP4 quantization, and a causal moving-mean cache supplies the activation mean during autoregressive decoding.

remain in higher precision, while each selected linear projection is evaluated with FP4 operands in the forward and backward matrix multiplications. We apply a fixed weight-decomposition path and FP4 quantization to weights, activations, and gradients; the activation and gradient quantization paths define the experimental condition.

Spectral Weight Factorization

We factor each selected weight matrix before post-training. Let $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and let its truncated rank- r singular-value decomposition be

$$W = U_r \Sigma_r V_r^\top + R_W, \quad R_W \triangleq W - U_r \Sigma_r V_r^\top, \quad (2)$$

where $U_r \in \mathbb{R}^{d_{\text{out}} \times r}$, $\Sigma_r \in \mathbb{R}^{r \times r}$, and $V_r \in \mathbb{R}^{d_{\text{in}} \times r}$. This decomposition follows the spectral motivation of Metis: a small number of large singular components can dominate the dynamic range of an otherwise quantization-friendly residual (Cao et al. 2025). In our implementation, $r = 64$ for the proposed condition. The low-rank right factor and the residual are evaluated through FP4 linear operators, whereas the left factor and singular values form the reconstruction path.

Denoting quantize-dequantize in FP4 by Q_{FP4} , the effective weight is

$$\widetilde{W} = U_r \Sigma_r Q_{\text{FP4}}(V_r^\top) + Q_{\text{FP4}}(R_W). \quad (3)$$

For a row-major activation matrix X , the corresponding forward GeMM is therefore $Y = \widetilde{X} \widetilde{W}^\top + b$. Equivalently, the implementation first applies the quantized $V_r^\top$ projection, scales the rank- r features by Σ_r , projects them with U_r , and adds the quantized residual branch. This form avoids forcing the dominant spectral component and the low-energy residual to share a single FP4 scale.

Causal Mean-Residual Quantization

The proposed condition uses mean-residual splitting for activations and output gradients. Let $X \in \mathbb{R}^{L \times d_{\text{in}}}$ denote a flattened activation matrix, where L merges batch and sequence positions. Following the mean-bias observation of Averis (Cao et al. 2026), we isolate the feature-wise mean

$$\mu_X = \frac{1}{L} \mathbf{1}_L^\top X, \quad X_R = X - \mathbf{1}_L \mu_X, \quad (4)$$

and apply FP4 only to the centered residual:

$$\hat{X} = \mathbf{1}_L \mu_X + Q_{\text{FP4}}(X_R). \quad (5)$$

Thus, the shared offset is restored in higher precision after the FP4 residual operation. Centering allocates the finite FP4 codebook to token-to-token variation rather than to a coherent rank-one offset. Although the implementation retains the generic Metis low-rank configuration interface, the moving-mean condition selects this mean-residual path for activations rather than computing an activation SVD at every step.

The backward output gradient is treated analogously. For $D = \partial \mathcal{L} / \partial Y \in \mathbb{R}^{L \times d_{\text{out}}}$, define

$$\begin{aligned} \mu_D &= \frac{1}{L} \mathbf{1}_L^\top D, & D_R &= D - \mathbf{1}_L \mu_D, \\ \hat{D} &= \mathbf{1}_L \mu_D + Q_{\text{FP4}}(D_R). \end{aligned} \quad (6)$$

The low-precision linear backward path then uses

$$\widehat{\nabla_W \mathcal{L}} = \hat{D}^\top \hat{X}, \quad \widehat{\nabla_X \mathcal{L}} = \hat{D} \tilde{W}. \quad (7)$$

Expanding the reconstructed low-precision weight-gradient GeMM yields four terms:

$$\begin{aligned} \widehat{\nabla_W \mathcal{L}} &= Q_{\text{FP4}}(D_R)^\top Q_{\text{FP4}}(X_R) + Q_{\text{FP4}}(D_R)^\top \mathbf{1}_L \mu_X \\ &\quad + \mu_D^\top \mathbf{1}_L^\top Q_{\text{FP4}}(X_R) + L \mu_D^\top \mu_X. \end{aligned} \quad (8)$$

Equation 8 is the form used by the FP4 implementation: it quantizes the residual streams, restores the mean components, and evaluates the resulting product without cancelling any term. Quantize–dequantize changes the residuals, and the causal activation mean and token masks need not match the gradient’s reduction domain. The computation therefore retains the residual–residual interaction, both mean–residual interactions, and the rank-one mean–mean interaction.

During prompt prefill, the method computes μ_X over all available prompt positions and stores both the mean and the number of represented token positions. At autoregressive decode step t , let μ_t be the mean of the newly observed positions and n_t their count. The cache is updated as

$$\mu_{\text{cache}} \leftarrow \frac{n_{\text{cache}} \mu_{\text{cache}} + n_t \mu_t}{n_{\text{cache}} + n_t}. \quad (9)$$

At decode time, Eq. 5 uses μ_{cache} in place of μ_X . This construction does not access future generated tokens and therefore matches the information available to an autoregressive decoder. A full-sequence training forward and a prefix-only decode forward can nevertheless use different means; we discuss this inherent train–decode mismatch in the conclusion.

Quantization-Aware Self-Distillation Before GRPO

The proposed pipeline has two stages. First, a frozen BF16 teacher samples a response y for each prompt x . The teacher and FP4 student then perform full forwards on the same concatenated sequence (x, y) . Let $p_T(\cdot \mid x, y_{<t})$ and $p_S(\cdot \mid x, y_{<t})$ be their temperature-scaled token distributions. We optimize only valid response positions:

$$\mathcal{L}_{\text{KD}} = T^2 \sum_{t \in y} \text{KL}(p_T(\cdot \mid x, y_{<t}) \parallel p_S(\cdot \mid x, y_{<t})). \quad (10)$$

Algorithm 1 QAT Self-Distillation Followed by FP4 GRPO

- 1: Initialize BF16 teacher π_T and FP4 student π_S from the same base checkpoint.
 - 2: **for** each QAT minibatch of prompts x **do**
 - 3: Sample teacher responses $y \sim \pi_T(\cdot \mid x)$.
 - 4: Update π_S with masked token-level distillation loss $\mathcal{L}_{\text{KD}}$.
 - 5: **end for**
 - 6: **for** each GRPO minibatch of prompts x **do**
 - 7: Sample quantized rollouts $\{y_i\}_{i=1}^G$ from π_S using the moving-mean cache.
 - 8: Compute task rewards and group-relative advantages.
 - 9: Update π_S with the GRPO objective.
 - 10: **end for**
-

To reduce the memory cost of the teacher signal, we retain the teacher’s top- k logits and gather the corresponding vocabulary entries from the student. This stage adapts the quantized computation path to the teacher’s response distribution before the policy is used to explore and optimize rewards. In the second stage, the resulting FP4 student is trained with GRPO on DeepMath-103K examples using a mathematical-equivalence reward.

Experiments

Models, Data, and Baselines

The study post-trains Qwen2.5-0.5B-Instruct (Yang et al. 2024a), Qwen2.5-Math-1.5B-Base, and Qwen2.5-Math-7B-Base (Yang et al. 2024b) on DeepMath-103K (He et al. 2025), a collection of 103,022 challenging mathematical problems curated for RL and supervised fine-tuning. The dataset provides a verifiable final answer for each problem, enabling rule-based rewards, and is constructed through benchmark decontamination and difficulty filtering. We use its problem prompts and final answers for GRPO; the QAT stage uses teacher-generated responses on the same prompt distribution.

The final matched comparison contains four conditions: (i) the pretrained checkpoint without post-training, (ii) BF16 GRPO, (iii) NV-Hadamard FP4 GRPO, and (iv) QAT plus moving-mean FP4 GRPO.

We use EvalScope to evaluate every reported checkpoint on AIME 2024, AIME 2025, AMC, MATH-500 (Lightman et al. 2023), ARC (Clark et al. 2018), and GPQA-Diamond (Rein et al. 2023). Each prompt receives one greedy completion with `do_sample=false`, and a maximum of 2048 newly generated tokens. We report the task accuracy produced by the corresponding EvalScope report.

Training Protocol

Table 1 specifies the default Qwen2.5-0.5B-Instruct diagnostic configuration encoded by the training entry points. Matched comparisons use the same DeepMath-103K training split, reward implementation, prompt order, generation limits, and GRPO budget. Direct FP4 disables the weight-decomposition and activation/gradient residual paths. QAT

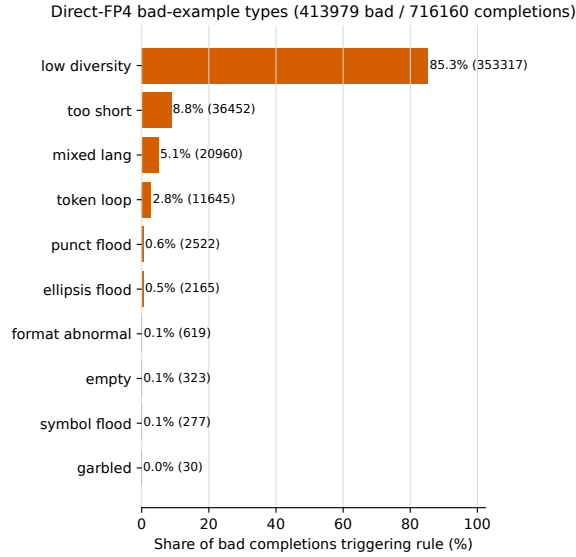


Figure 2: Multi-label composition of direct-FP4 bad rollouts for Qwen2.5-0.5B-Instruct. A completion can receive more than one failure label.

plus moving mean uses rank-64 weight decomposition and mean bias activation/gradient residual paths.

Rollout Diagnostics

Scalar benchmark accuracy is insufficient to explain why a post-training run succeeds or fails. The system therefore records per-completion and per-step text-quality statistics: empty and short outputs, non-printable-character ratio, repeated n -grams, consecutive repeated tokens, lexical diversity, punctuation and symbol ratios, and multi-language switching. It aggregates a rule-based bad-rollout rate, reward statistics, and separate trends for responses classified as good and bad. Complementary diagnostics record activations and rollout-related probability summaries. The bad-rollout label is multi-label: a completion can be assigned more than one failure type.

Results

QAT Stabilizes FP4 Rollouts Before GRPO

We first ask whether a BF16 policy can be used directly under the FP4 execution path to generate the trajectories required by GRPO. Figure 2 decomposes the direct-FP4 failures behind the aggregate bad-rollout rate. Low lexical diversity and explicit token loops account for a large fraction of flagged completions, with short outputs and mixed-language generations providing additional evidence of decoding degradation. Because the categories are assigned independently, their proportions need not sum to one. This decomposition is important: it distinguishes a high bad-rollout rate caused by visible generation failures from a mere change in task difficulty or reward calibration.

Figure 3 compares the bad-rollout rate of direct FP4 GRPO and QAT plus moving-mean FP4 on the available Qwen2.5-

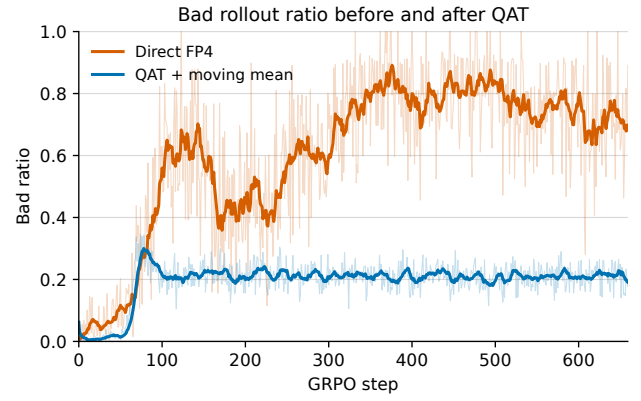


Figure 3: Bad-rollout-rate trajectories for Qwen2.5-0.5B-Instruct. Direct FP4 is compared with QAT plus moving-mean FP4 over the logged GRPO steps. Pale traces are logged values and solid traces are trailing moving averages.

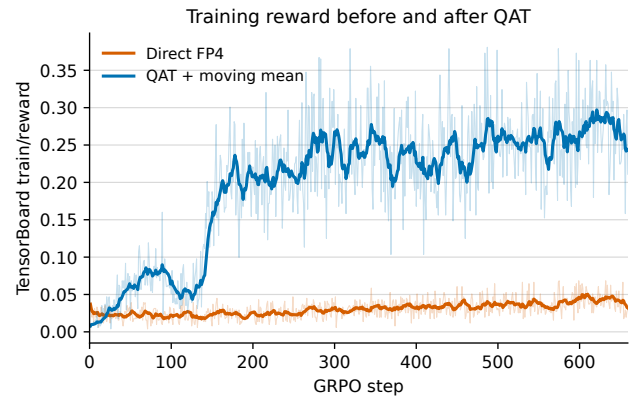


Figure 4: Mean training-reward trajectories for Qwen2.5-0.5B-Instruct. The plot uses TensorBoard `train/reward`; pale traces are logged values and solid traces are trailing moving averages.

0.5B-Instruct diagnostic run. Direct FP4 has a consistently higher bad-rollout rate, whereas QAT pre-adaptation substantially reduces this rate throughout the observed training trajectory.

Figure 4 shows the corresponding TensorBoard `train/reward` signal. It remains low for direct FP4 but rises to a materially higher level after QAT. Together, the two trajectories support the proposed ordering of the pipeline: adapt the quantized policy before using its samples for reward optimization.

Main Results

Table 2 summarizes the downstream comparison across model scales and task families. QAT with moving-mean quantization enables substantial post-training gains for the math-specialized models. Relative to their pretrained checkpoints, the QAT policies improve the 1.5B and 7B models by large margins on AMC, MATH-500, ARC, and GPQA-

Table 1: Default controlled diagnostic protocol for Qwen2.5-0.5B-Instruct. QAT and GRPO step budgets are set explicitly for each reported run.

Setting	QAT self-distillation	GRPO post-training
Data	DeepMath-103K training split; identical prompt order	within each seed
Precision	BF16 teacher; FP4 student forward/backward	BF16 master parameters; FP4 policy forward/backward
Batching	16 prompts/GPU; accumulation 4	64 prompts/GPU; accumulation 1; $G = 8$ completions/prompt
Sequence limits	prompt 512; teacher generation 1024	prompt 1024; completion 1024
Optimization	AdamW, learning rate 10^{-6} , 5% warmup, gradient norm 1.0	AdamW, learning rate 10^{-6} , gradient norm 1.0
Distillation	$T = 1$, one teacher response/prompt, top- $k = 100$	group-relative accuracy reward
Generation	—	cache enabled; gradient checkpointing enabled
FP4 operator	moving-mean residual, according to condition	same operator as the corresponding condition
Residual ranks	weight: 64 when enabled	weight: 64 when enabled

Diamond; the 7B policy also improves both AIME evaluations. The QAT policies remain close to their BF16-GRPO counterparts on most mathematical tasks. For the 1.5B model, the largest gap is 6.0 points on AMC, while MATH-500 differs by only 2.4 points and ARC is slightly higher under QAT. For the 7B model, QAT matches BF16 on MATH-500 and exceeds it on AIME24, AIME25, and AMC. The principal exception is ARC, where the 7B QAT policy trails BF16 by 18.5 points.

Table 2: Downstream accuracy (%) evaluated with EvalScope.

Model	Method	AIME24	AIME25	AMC	MATH-500	ARC	GPQA-Diamond
Qwen2.5-0.5B	Pretrained	0.0	0.0	6.72	29.6	54.62	18.69
	BF16 GRPO	3.3	0.0	8.95	32.2	53.22	24.24
	NV-Hadamard GRPO	—	—	—	—	—	—
	QAT + moving mean	0.0	0.0	6.72	6	29.6	24.24
Qwen2.5-Math-1.5B-Base	Pretrained	6.67	10.0	20.15	25.8	7.89	0.51
	BF16 GRPO	13.3	3.3	38.8	70.4	61.61	22.73
	NV-Hadamard GRPO	—	—	—	—	—	—
	QAT + moving mean	10.0	6.67	32.8	68.0	63.39	19.7
Qwen2.5-Math-7B-Base	Pretrained	6.67	3.3	5.97	17.4	15.53	8.59
	BF16 GRPO	10.0	3.3	46.27	78.0	78.52	33.84
	NV-Hadamard GRPO	—	—	—	—	—	—
	QAT + moving mean	13.3	13.3	50.74	78.0	60.07	31.31

Table 3: GRPO reward summary across model scales.

Model	Method	Start r	Final r
Qwen2.5-0.5B	BF16 GRPO	0.1130	0.3066
Qwen2.5-0.5B	Direct FP4 GRPO	0.0200	0.0398
Qwen2.5-0.5B	QAT + moving mean	0.0100	0.2316
Qwen2.5-Math-1.5B-Base	BF16 GRPO	0.3750	0.4378
Qwen2.5-Math-1.5B-Base	Direct FP4 GRPO	0.0820	0.0191
Qwen2.5-Math-1.5B-Base	QAT + moving mean	0.0664	0.4023
Qwen2.5-Math-7B-Base	BF16 GRPO	0.1992	0.4448
Qwen2.5-Math-7B-Base	Direct FP4 GRPO	0.1641	0.2949
Qwen2.5-Math-7B-Base	QAT + moving mean	0.1660	0.3379

The 0.5B instruction model exhibits a different regime: QAT does not retain the BF16 gains on MATH-500 or ARC and falls below its pretrained score on both tasks. Taken together, the results show that the proposed pipeline can preserve strong reasoning performance at FP4 for math-specialized backbones, while robustness remains sensitive to model scale and task family.

Training-Signal Comparison. Table 3 reports the first and final logged GRPO rewards for each model and precision condition. These endpoints expose whether the policy obtains a useful learning signal under the FP4 rollout path.

Direct FP4 produces the weakest final reward at every scale. Its failure is most pronounced at 0.5B and 1.5B: reward rises only from 0.0200 to 0.0398 for the former and falls from 0.0820 to 0.0191 for the latter. QAT plus moving mean changes this behavior. It raises the 0.5B reward to 0.2316 and the 1.5B reward to 0.4023, approaching the corresponding BF16 values of 0.3066 and 0.4378. At 7B, both FP4 variants improve during GRPO, but QAT plus moving mean finishes higher than direct FP4 (0.3379 versus 0.2949), while BF16 remains highest at 0.4448. The improvement is therefore not simply a consequence of a favorable starting reward: the two FP4 conditions begin at comparable levels for the 0.5B and 7B models, whereas their final rewards separate after training. Combined with the rollout diagnostics, this pattern supports QAT as an effective preparation step for obtaining

a usable FP4 GRPO learning signal.

Empirical Takeaway

The results form a consistent picture of FP4 post-training. Direct FP4 degrades rollout quality and yields the weakest final reward across all three model scales, demonstrating that low-precision execution can disrupt the on-policy data on which GRPO depends. QAT self-distillation addresses this failure before reward optimization: it substantially improves the FP4 reward trajectory and enables strong downstream reasoning performance for the math-specialized 1.5B and 7B models. The remaining gap on selected tasks, most clearly ARC for the 7B model and MATH-500 for the 0.5B instruction model, identifies robustness across model families as the central challenge for FP4 post-training.

Conclusion

FP4 post-training cannot be evaluated solely as a low-precision optimization problem because an RL policy generates the trajectories from which it learns. We presented a two-stage FP4 pipeline that first adapts a quantized student to a BF16 teacher through masked self-distillation and then applies GRPO with causal moving-mean residual quantization. This ordering targets the rollout distribution before reward optimization closes the feedback loop.

Across three model scales, direct FP4 produces degraded rollout quality and the weakest final reward. QAT plus moving mean substantially improves the FP4 learning signal and preserves strong downstream reasoning performance for the 1.5B and 7B math-specialized models. These findings establish rollout quality as a central criterion for FP4 RL post-training: a quantized policy must generate usable candidate groups, not merely execute low-precision matrix multiplications on a fixed batch.

The remaining challenges clarify the scope of the approach. The causal moving-mean cache can differ from a full-sequence training forward, and the rule-based rollout classifier captures visible degeneration rather than every form of semantic failure. Moreover, the ARC and 0.5B results show that stable FP4 optimization does not guarantee uniform accuracy across all model families and tasks. Reporting

the complete quantization recipe, decoding configuration, hardware and software environment, and seed-level variation is therefore essential for reproducible FP4 post-training research.

References

- Ashkboos, S.; Mohtashami, A.; Croci, M. L.; Li, B.; Cameron, P.; Jaggi, M.; Alistarh, D.; Hoefler, T.; and Hensman, J. 2024. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. arXiv:2404.00456.
- Cao, H.; Chen, M.; Yang, Y.; Huang, R.; Dong, F.; Zhou, J.; Chen, A.; Dong, M.; Wang, Y.; Hou, J.; Cheng, Y.; Wu, F.; Yang, F.; Lu, T.; Gu, N.; and Shang, L. 2025. Metis: Training LLMs with FP4 Quantization. arXiv:2509.00404.
- Cao, H.; Huang, Z.; Chen, M.; Yang, Y.; Dong, F.; Chen, A.; Huang, R.; Zhang, X.; Dong, M.; Wang, Y.; Hou, J.; Lv, Q.; Dick, R. P.; Cheng, Y.; Lu, T.; Yang, F.; Chen, Y.; and Shang, L. 2026. The Curse and Blessing of Mean Bias in FP4-Quantized LLM Training. arXiv:2603.10444.
- Chmiel, B.; Fishman, M.; Banner, R.; and Soudry, D. 2025. FP4 All the Way: Fully Quantized Training of LLMs. arXiv:2505.19115.
- Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Taffjord, O. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457.
- Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. LLM.int8(): 8-Bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339.
- Du, D.; Zhang, Y.; Cao, S.; Guo, J.; Cao, T.; Chu, X.; and Xu, N. 2024. BitDistiller: Unleashing the Potential of Sub-4-Bit LLMs via Self-Distillation. arXiv:2402.10631.
- Esser, S. K.; McKinstry, J.; Bablani, D.; Appuswamy, R.; and Modha, D. S. 2020. Learned Step Size Quantization. In *International Conference on Learning Representations*.
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations*.
- He, Z.; Liang, T.; Xu, J.; Liu, Q.; Chen, X.; Wang, Y.; Song, L.; Yu, D.; Liang, Z.; Wang, W.; Zhang, Z.; Wang, R.; Tu, Z.; Mi, H.; and Yu, D. 2025. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning. arXiv:2504.11456.
- Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531.
- Huang, W.; Ge, Y.; Yang, S.; Xiao, Y.; Mao, H.; Lin, Y.; Ye, H.; Liu, S.; Cheung, K. C.; Yin, H.; Lu, Y.; Qi, X.; Han, S.; and Chen, Y. 2025. QeRL: Beyond Efficiency – Quantization-enhanced Reinforcement Learning for LLMs. arXiv:2510.11696.
- Lee, D.; Choi, S.; and Chang, I. J. 2025. QRazor: Reliable and Effortless 4-bit LLM Quantization by Significant Data Razoring. arXiv:2501.13331.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s Verify Step by Step. arXiv:2305.20050.
- Lin, J.; Tang, J.; Tang, H.; Yang, S.; Dang, X.; and Han, S. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*.
- Liu, S.-y.; Liu, Z.; Huang, X.; Dong, P.; and Cheng, K.-T. 2023. LLM-FP4: 4-Bit Floating-Point Quantized Transformers. arXiv:2310.16836.
- NVIDIA; Abecassis, F.; Agrusa, A.; et al. 2025. Pretraining Large Language Models with NVFP4. arXiv:2509.25149.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Shao, W.; Chen, M.; Zhang, Z.; Xu, P.; Zhao, L.; Li, Z.; Zhang, K.; Gao, P.; Qiao, Y.; and Luo, P. 2024a. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. arXiv:2308.13137.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024b. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Sun, Y.; Liu, R.; Bai, H.; Bao, H.; Zhao, K.; Li, Y.; Hu, J.; Yu, X.; Hou, L.; Yuan, C.; Jiang, X.; Liu, W.; and Yao, J. 2024. FlatQuant: Flatness Matters for LLM Quantization. arXiv:2410.09426.
- Wang, R.; Gong, Y.; Liu, X.; Zhao, G.; Yang, Z.; Guo, B.; Zha, Z.; and Cheng, P. 2025. Optimizing Large Language Model Training Using FP4 Quantization. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438.
- Xin, M.; Priyadarshi, S.; Xin, J.; Kartal, B.; Vavre, A.; Thekkumpate, A. K.; Chen, Z.; Mahabaleshwarkar, A. S.; Shahaf, I.; Bercovich, A.; Patel, K.; Velury, S. V.; Luo, C.; Cheng, Z.; Chen, J.; Yu, C.-H.; Ping, W.; Rybakov, O.; Tajbakhsh, N.; Olabi, O.; Stosic, D.; Wu, D.; Han, S.; Chung, E.; Sreenivas, S. T.; Catanzaro, B.; Suhara, Y.; Blankevoort, T.; and Mao, H. 2026. Quantization-Aware Distillation for NVFP4 Inference Accuracy Recovery. arXiv:2601.20088.
- Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; et al. 2024a. Qwen2.5 Technical Report. arXiv:2412.15115.
- Yang, A.; Zhang, B.; Hui, B.; Gao, B.; Yu, B.; Li, C.; Liu, D.; Tu, J.; Zhou, J.; Lin, J.; Lu, K.; Xue, M.; Lin, R.; Liu, T.; Ren, X.; and Zhang, Z. 2024b. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv:2409.12122.